

Solving Multiphysics and Multiscale Problems with AI-Augmented Simulations (CFD-PINN)

Part 1: Introduction to Fundamentals

Jeff Gomes

(Twitter/X: @CoPhyMs)



XXIX ENMC
ENCONTRO NACIONAL DE MODELAGEM COMPUTACIONAL

XVII ECTM
ENCONTRO DE CIÊNCIA E TECNOLOGIA DE MATERIAS

Section Outline

- 1 History & Motivation for PINNs
- 2 Let's Briefly Talk about PDEs – From a difference quotient to the Cauchy problem
- 3 Introduction to CFD

Physics Informed Neural Networks: why?

- a Classical solvers (FEM, FVM, spectral methods) discretise space and time and march the physics forward, mesh cell by mesh cell.
- b PINNs instead encode the physics — the governing PDE, boundary and initial conditions — directly into the **loss function** of a neural network.
- c The network is trained to minimise the residual of the equations everywhere in the domain, not just at grid points.
- d Solving a physical problem becomes a hybrid of **physics** (the equations) and **data** (measurements, when available).

Takeaway

PINNs are an additional tool for solving (P)DEs — not a replacement for classical solvers.

They face difficulties classical solvers do not, but many can be tackled with insights from deep learning.

PINNs work best when they are actually *data-informed*.

ORIGINS

Physics Informed Neural Networks: a historical note



1998 — Lagaris, Likas & Fotiadis

First explicit method to solve initial and boundary value problems using artificial neural networks.



2001 — Aarts & Van Der Veer

Solve PDEs and their boundary/initial conditions with single-hidden-layer feedforward networks.



2011 — Kumar & Yadav (survey)

A survey of MLP and radial-basis-function network methods for solving differential equations.



2017–2019 — Raissi, Perdikaris & Karniadakis

Coin “Physics-Informed Deep Learning”; formalise forward/inverse problems using automatic differentiation.

Section Outline

- 1 History & Motivation for PINNs
- 2 Let's Briefly Talk about PDEs – From a difference quotient to the Cauchy problem**
- 3 Introduction to CFD

What is a differential equation?

- An equation relating an unknown function to its own derivatives.
- An **ODE** involves derivatives with respect to a single variable (e.g. time); a **PDE** involves partial derivatives with respect to several variables (e.g. space *and* time).
- Almost every conservation law in physics and engineering — heat, momentum, mass, charge — is naturally written as a PDE.
- Solving one means finding a function, not a number: the unknown lives in an infinite-dimensional space of functions.

A simple example

$$\frac{du}{dt} = -k u(t)$$

Exponential decay: the rate of change of u is proportional to u itself — the unknown is the function $u(t)$, not a single number.

RECAP

Differentiation and Taylor expansion

- For a continuous f , the **difference quotient** $\frac{f(x+h) - f(x)}{h}$ measures average change over h .
- f is **differentiable** at x if this quotient has a finite limit as $h \rightarrow 0$: that limit is $f'(x)$.
- Iterating gives higher-order derivatives $f^{(n)}$, provided each one exists.
- The **Taylor expansion** rebuilds f near a point entirely from its derivatives there:

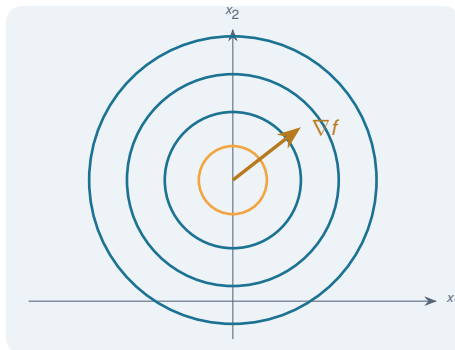
$$f(x+h) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!} h^n$$

The whole function is encoded in the sequence of its derivatives at a single point — the first hint that “function” and “sequence of numbers” are closer than they seem.

RECAP

Partial derivatives and the gradient

- For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, expand along one coordinate direction at a time: the **partial derivative** $\partial f / \partial x_i$ freezes every other variable.
- Stacking all partials gives the **gradient** $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$, a vector pointing in the direction of steepest increase.
- Second partials build the **Hessian**; mixed partials appear constantly in PDEs (e.g. $\partial^2 u / \partial x \partial t$).
- This is exactly the machinery automatic differentiation will later compute for us, exactly, inside a neural network.



Level sets of f ; ∇f is normal to them, pointing toward higher values

Standard equations vs. differential equations

System of equations	System of differential equations
Unknown: a vector $x \in \mathbb{R}^n$	Unknown: a function $u(\cdot)$ in a function space
$F(x) = 0$	$\mathcal{N}[u] = 0$, with $\mathcal{N}$ a differential operator
If F is linear: $Ax = b$	If $\mathcal{N}$ is linear: a linear PDE
Solved by linear algebra	Solved by calculus of variations, spectral methods, or — now — neural networks

If the operator is linear, we inherit all the superposition machinery of linear algebra — just in infinite dimensions.

The Cauchy problem

- A PDE alone has many solutions (depending on ICs and BCs).
- The pair (**Differential Equation, Boundary/Initial Conditions**) is called a **Cauchy problem**.
- The **Cauchy–Kovalevskaya theorem** guarantees existence and uniqueness of a (locally analytic) solution for a well-posed Cauchy problem.
- Every solver (classical or neural) attempts to solve a Cauchy problem, whether or not it says so explicitly.

$$\mathcal{N}[u](x) = f(x), \quad x \in \Omega$$

$$u(x) = g(x), \quad x \in \partial\Omega$$

Together, the governing equations on the domain Ω and the conditions on its boundary $\partial\Omega$ define the Cauchy problem.

CAUCHY

The general form of a differential equation

$$\mathcal{N}_\lambda[u](x) = f(x), \quad x \in \Omega \subset \mathbb{R}^d$$

$\mathcal{N}_\lambda$: differential operator, parameterised by physical constants λ (diffusivity, viscosity etc)

f : source term; u : unknown field

Geometry

- The domain Ω and its boundary $\partial\Omega$.
- The prescribed values of u (or its derivatives) on $\partial\Omega$ — the BC/IC.

Physics

- The differential operator $\mathcal{N}_\lambda$ acting on the field u , governed by physical parameters λ .
- The source term f .

CLASSIFICATION

A few examples of PDEs

Elliptic

Laplace: $\nabla^2 u = 0$

Poisson: $\nabla^2 u = f$

Darcy flow

Helmholtz PDE

Parabolic

Heat: $u_t = \alpha \nabla^2 u$

Burgers equation

Schrödinger PDE

Fokker–Planck

Advection–diffusion

Hyperbolic

Wave: $u_{tt} = c^2 \nabla^2 u$

Advection/Transport

Classification (2D, second order): elliptic $B^2 - 4AC < 0$, parabolic $= 0$, hyperbolic > 0 .

CLOSURE

Initial and boundary conditions

- A second-order linear PDE alone is not enough; we need conditions on $\partial\Omega$ (and at $t = 0$, if time-dependent) to close the problem.
- These conditions come in three classical flavours, and mixing them across different parts of $\partial\Omega$ is common in real applications.



Dirichlet

Prescribes the value of u on the boundary.



Neumann

Prescribes the normal gradient $\partial u / \partial n$ on the boundary.



Robin

A linear combination of Dirichlet and Neumann conditions.

Section Outline

- 1 History & Motivation for PINNs
- 2 Let's Briefly Talk about PDEs – From a difference quotient to the Cauchy problem
- 3 Introduction to CFD**

Conservation laws for fluid flow

- Computational Fluid Dynamics (CFD) solves the equations that express conservation of **mass**, **momentum**, and **energy** for a moving fluid.
- These are the **Navier–Stokes equations (NSE)**, a coupled, nonlinear system of PDEs, parabolic-hyperbolic in character.
- The nonlinear advection term $(\mathbf{u} \cdot \nabla)\mathbf{u}$ is the source of most of CFD's difficulty and of turbulence itself.
- No general closed-form solution exists.

Incompressible Navier–Stokes

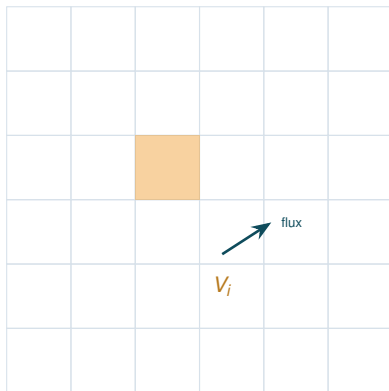
$$\nabla \cdot \mathbf{u} = 0$$

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}$$

Mass conservation (continuity) + momentum balance: inertia, pressure, viscous diffusion, body forces.

Discretising the domain

- A PDE holds at every point of a continuous domain — a computer can only store finitely many numbers.
- **Discretisation** replaces the domain with a mesh of cells, faces, or nodes, and the continuous fields with values at those locations.
- The **Finite Volume Method (FVM)** — dominant in commercial CFD — integrates the conservation laws over each control volume, guaranteeing exact conservation cell by cell.
- Finite Difference and Finite Element methods offer alternative discretisations, each with its own trade-offs.



*A structured finite-volume mesh:
physics is enforced cell by cell*

Turbulence and the closure problem

- Most flows of engineering interest are **turbulent**: chaotic, multi-scale, sensitive to initial conditions.
 - **Direct Numerical Simulation (DNS)**
resolves every scale down to the Kolmogorov length — accurate, but its cost scales as $Re^{9/4}$: infeasible for most real geometries.
 - **RANS** models the mean flow and models turbulence statistically ($k-\epsilon$, $k-\omega$); **LES** resolves large eddies and models only the small ones.
 - Averaging introduces new unknowns (the Reynolds stresses) that themselves need modelling — the **closure problem**.
- **DNS**
All scales resolved — gold standard, extreme cost.
 - **LES**
Large eddies resolved, small ones modelled.
 - **RANS**
Only the mean flow resolved; all turbulence modelled.

Why CFD is so expensive — and why this course exists

- A single high-fidelity CFD run can take hours to weeks, even on HPC clusters — and design, optimisation, and uncertainty quantification need *thousands* of such runs.
- Re-meshing for every new geometry is itself a costly, manual bottleneck.
- This is precisely the motivation from Part I: if a trained network can approximate the solution field directly, inference costs milliseconds instead of hours.
- Understanding classical CFD is essential before understanding *where* and *why* a PINN or a learned surrogate can help — and where it cannot (yet) replace a solver.

Cost, one design iteration

Hours – weeks

classical high-fidelity CFD solve

Milliseconds

trained neural surrogate, once trained